Kubernetes

Metrics Serverで
リソース監視の基盤を作る

kubectl top とオートスケーリングを支える仕組み

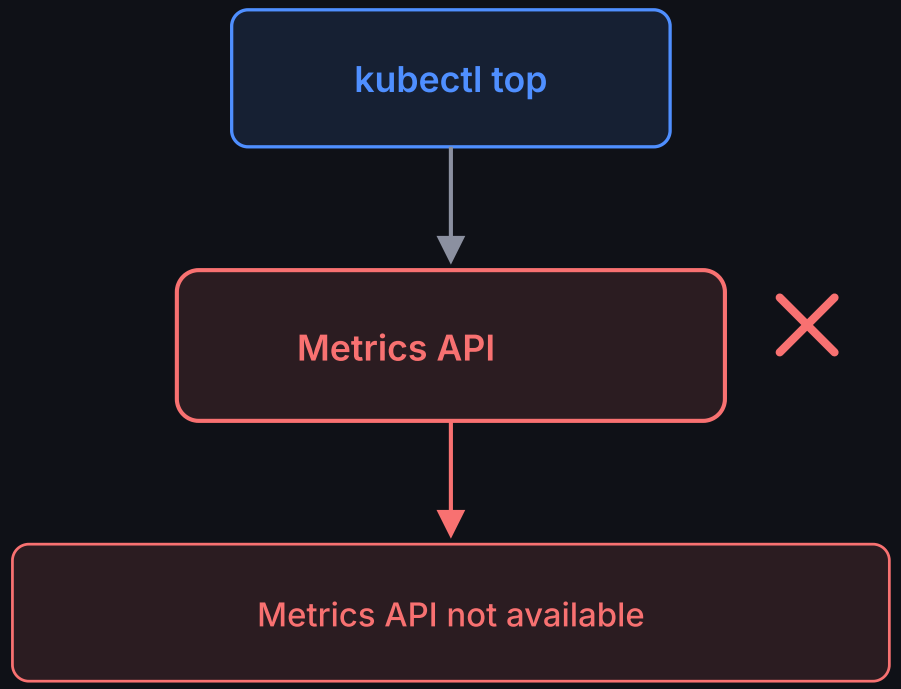
viserhaut — 2026

SECTION 01

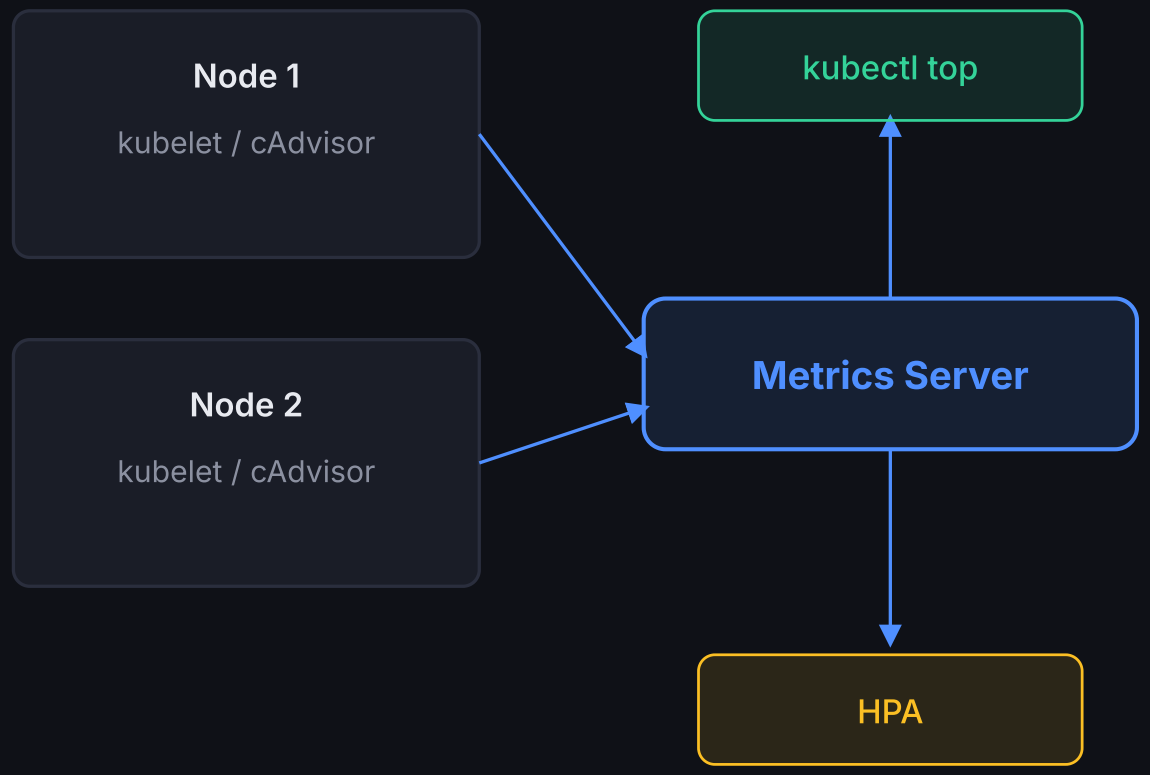
なぜMetrics Serverが必要か

kubectl topが使えないのはメトリクス収集基盤がないから

- **kubectl top** 実行時に "Metrics API not available" エラーが発生する
- HPAがメトリクスを取得できず、**オートスケーリングが機能しない**
- ノードやPodのリソース使用量が**可視化できず**、障害対応が遅れる



Metrics Serverがノード全体のメトリクスを集約する



- 各ノードの kubelet からCPU/メモリの使用量を定期的に収集する
- 集約したメトリクスを Metrics API として公開し、kubectl top や HPA が参照する
- cAdvisor が個別ノード内で収集したデータをクラスタ全体に配信する役割を持つ

EKS vs GKE — Metrics Serverの扱いの違い

観点	EKS	GKE
デフォルト状態	未インストール	自動インストール済み
導入方法	手動でデプロイが必要	設定不要ですぐ使える
設計思想	汎用性重視で自由度が高い	マネージド統合で手間を削減

Minikubeへの導入とkubectl topの実行

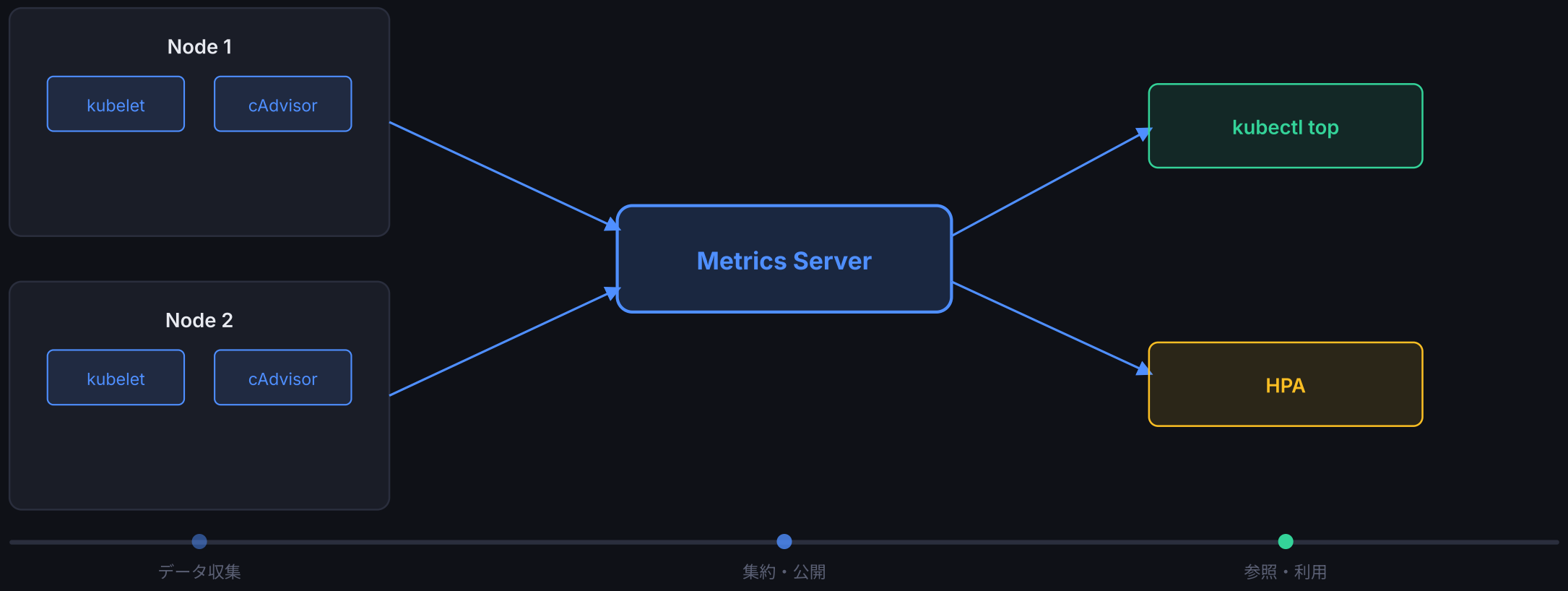
```
# Metrics Serverの有効化
minikube addons enable metrics-server

# 動作確認(2-3分待ってから実行)
kubectl get pods -n kube-system | grep metrics

# ノードのリソース使用量を確認
kubectl top nodes

# Pod単位のリソース使用量を確認
kubectl top pods -A
```

addons enable
MinikubeではAddonで簡単に導入できる

get pods
kube-system名前空間にMetrics ServerのPodが起動する

top pods -A
全名前空間のPod使用量を一覧表示

cAdvisor → Metrics Server → kubectl top / HPA の全体像



Metrics Server導入の3ステップ

01

Metrics Serverをデプロイする

EKSでは手動インストール、GKEではデフォルト有効。Minikubeではaddonで導入

02

kubectl topで可視化する

ノードとPodのCPU/メモリ使用量をリアルタイムに確認できるようになる

03

HPAの土台として活用する

Metrics Serverが提供するメトリクスがHPAのスケーリング判断の基準になる